

Giraffe

*A LangGraph-Powered Multi-Agent Orchestration System
for Distributed AI Coding Agents*

Student	Yahya Efe Kuruçay
Student No	20220808005
Course	CSE 336 — Advanced Web Programming
Date	April 20, 2026
Email	contact@efekurucay.com
Website	efekurucay.com
Repository	github.com/GiraffeGraph/giraffe
Technology	LangGraph · Next.js · TypeScript

1. Project Overview
2. Why LangGraph?
3. Architecture: Smart Orchestrator, Dumb Agents
4. LangGraph State Machine — Implementation Deep Dive
5. State Schema
6. Graph Nodes
7. Conditional Routing (Edges)
8. Graph Assembly & Compilation
9. Database Schema for Agents
10. API Surface
11. Full-Stack Integration (Next.js)
12. Features Implemented
13. Setup & Running the Project
14. Git & Version Control
15. Reflection & Discussion Points

1 Project Overview

Giraffe is a self-hosted knowledge editor and distributed multi-agent orchestration platform built with Next.js 16, React 19, PostgreSQL, and TypeScript. The core idea is twofold:

- A **block-based knowledge editor** (similar to Notion) with wikilinks, backlinks, and graph navigation — all stored canonically in PostgreSQL.
- A **LangGraph-powered orchestration engine** that coordinates multiple AI coding agents (Claude Code, Aider, etc.) running in terminal sessions on remote machines.

The LangGraph component — the focus of this assignment — is the "Giraffe Orchestrator": a stateful state machine that plans work and dispatches tasks across a registry of AI agents, collecting handoff notes between them to preserve context.

Layer	Technology
Framework	Next.js 16 (App Router)
UI	React 19 + Tiptap editor
Database	PostgreSQL 16 via Prisma 7
Auth	NextAuth (credential-based)
AI Orchestration	LangGraph (@langchain/langgraph ^1.2.9)
LLM Client	AI SDK — @ai-sdk/openai, generateText
Agent Communication	tmux (capture-pane, send-keys) over SSH
Deployment	Docker Compose + nginx

2 Why LangGraph?

Coordinating multiple AI agents that must hand work off to each other sequentially — while each can succeed, fail, or time out — is a classic stateful workflow problem. A simple chain of function calls would not handle:

- **Branching:** what to do when the planner returns an empty plan or invalid JSON?
- **Loops:** re-entering the same node (`runStep`) once per plan step.
- **State persistence:** every node sees the full, typed state and can update it atomically.
- **Abort / cancellation:** passing an `AbortSignal` from the HTTP layer down to the graph invocation.

LangGraph provides exactly these primitives: typed state annotations, nodes as pure async functions, and conditional edges that decide the next node at runtime. The result is a graph that is easy to reason about as a diagram and equally easy to extend.

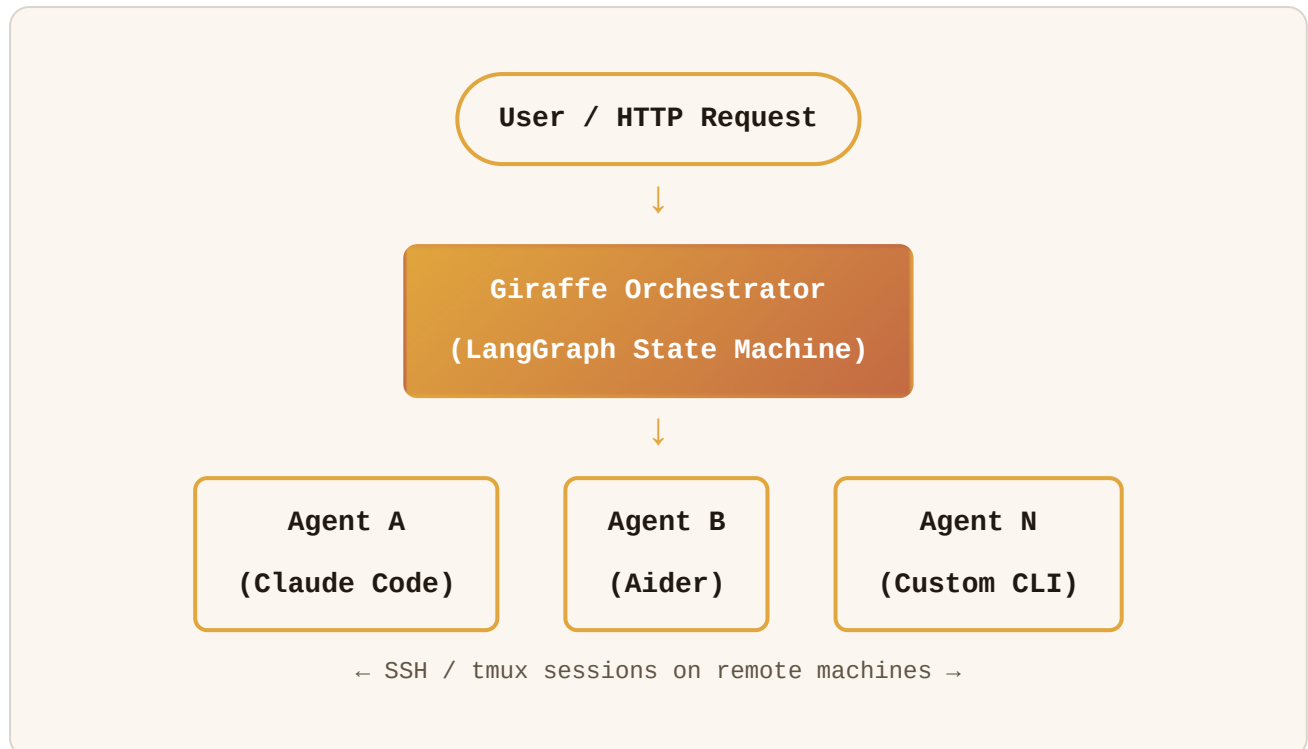
3 Architecture: Smart Orchestrator, Dumb Agents

The architectural philosophy behind the orchestrator is deliberately asymmetric:

- **Smart Orchestrator** — the LangGraph state machine. It uses an LLM (GPT-4o by default) to decompose a high-level goal into an ordered list of steps, deciding which agent handles each step. It manages timing, timeouts, handoffs, and persistence.

- **Dumb Agents** — existing CLI tools (Claude Code, Aider, custom scripts) that run in terminal sessions. The orchestrator does *not* set their model or system prompt; those are baked into the CLI tool's own configuration. The orchestrator only types text into the terminal and waits for the idle marker (e.g., the shell prompt `>`).

This separation means any terminal-based AI coding tool can be a "dumb agent" without modification. New agent types can be registered through the UI by specifying their start command and idle marker.



4 LangGraph State Machine — Implementation Deep Dive

The full source lives at `src/lib/orchestrator.ts`. The graph has two nodes:



↓

END

The graph allows up to **64 recursive steps** (the `recursionLimit`) which supports sessions with up to ~30 agent tasks before a hard stop.

5 State Schema

LangGraph uses `Annotation.Root` to define typed state. Each field declares a *reducer* (how new values are merged) and a default. Fields without an explicit reducer replace the previous value.

```
import { StateGraph, Annotation, END, START } from "@langchain/langgraph";

const OrchestratorState = Annotation.Root({
  // Session identity
  sessionId: Annotation<string>(),
  goal: Annotation<string>(),
  workingDirectory: Annotation<string>(),
  supervisorModel: Annotation<string>(),

  // Agent registry – replace-on-write
  agentRegistry: Annotation<AgentInfo[]>({
    reducer: (_, y) => y,
    default: () => [],
  }),

  // Plan produced by the planner node – replace-on-write
  plan: Annotation<PlanStep[]>({
    reducer: (_, y) => y,
    default: () => [],
  }),

  currentStepIndex: Annotation<number>({ reducer: (_, y) => y, default: () => 0
  }),
  handoffPayload: Annotation<string>({ reducer: (_, y) => y, default: () => ""
  }),
  phase: Annotation<string>({ reducer: (_, y) => y, default: () => "planning" }),
  aborted: Annotation<boolean>({ reducer: (_, y) => y, default: () => false }),
  error: Annotation<string>({ reducer: (_, y) => y, default: () => ""
  }),
});
```

Each `PlanStep` carries:

Field	Type	Purpose
<code>agentId</code>	string	Database ID of the assigned agent
<code>agentLabel</code>	string	Human-readable agent name
<code>task</code>	string	Natural language task description
<code>status</code>	pending executing done failed	Live progress tracking
<code>handoffNote</code>	string	Context note written by the agent for the next one

6 Graph Nodes

6.1 — `planner` Node

The planner node calls an OpenAI model with the session goal and the list of available agents. It asks the model to return a raw JSON array of steps — no markdown fences allowed. The response is cleaned and parsed; on failure the phase is set to `"failed"` and the graph ends immediately via conditional routing.

```
async function plannerNode(state: OState): Promise<Partial<OState>> {
  const { text } = await generateText({
    model: openai(supervisorModel || "gpt-4o"),
    system: `You are an orchestrator that plans tasks for AI coding agents in terminals.`,
    prompt: `Goal: ${goal}\nAvailable agents:\n${agentDescriptions}\n
    Return a JSON array. Each step:
    { "agentId": "<id>", "agentLabel": "<label>", "task": "<description>" }
    Start with [ and end with ].`,
    temperature: 0.1,
  });

  const plan = JSON.parse(text.trim()).map(step => ({
    ...step, status: "pending", handoffNote: ""
  }));

  await updateAgentSessionPlan(sessionId, plan);
  return { plan, currentStepIndex: 0, phase: "executing" };
}
```

6.2 — `runStep` Node

The most complex node. For each step it:

1. Looks up the target agent in `agentRegistry`.
2. Prepends the previous agent's handoff note to the task message (if any).
3. Writes the task to the agent's tmux terminal via `sendToAgentInput()`.
4. Polls `waitForIdleMarker()` — checking that the agent's configured idle marker appears in terminal output — with a 5-minute timeout.
5. Captures the terminal output via `captureTmuxPane()` and persists it as an `AgentMessage`.
6. If more steps remain, asks the current agent to write a `[HANDOFF_START]...[HANDOFF_END]` note, waits for idle again, and extracts the note.
7. Marks the step as `"done"` and advances `currentStepIndex`.

```
async function runStepNode(state: OState): Promise<Partial<OState>> {
  const { sendToAgentInput, waitForIdleMarker,
    extractHandoffNote, captureTmuxPane } =
    await import("@/lib/ws-terminal-server");

  // 1. Send task (with handoff from previous agent if any)
  const taskMessage = handoffPayload
    ? `Previous agent's handoff:\n${handoffPayload}\n\n---\nYour task: ${step.task}`
    : step.task;
  await sendToAgentInput(agent.id, taskMessage);

  // 2. Wait for idle (5-minute timeout)
  const isIdle = await waitForIdleMarker(agent.id, agent.idleMarker, 300_000);
  if (!isIdle) return { phase: "failed", error: "Agent timed out" };

  // 3. Capture output
  const agentOutput = await captureTmuxPane(agent.id, 100);

  // 4. Request handoff note if more steps remain
  if (!isLastStep) {
    await sendToAgentInput(agent.id, handoffPrompt);
    await waitForIdleMarker(agent.id, agent.idleMarker, 120_000);
    newHandoffPayload = await extractHandoffNote(agent.id);
  }

  return { plan: updatedPlan, currentStepIndex: nextIndex, handoffPayload: newHandoffPayload };
}
```

7

Conditional Routing (Edges)

LangGraph's conditional edges call a *router function* after each node completes. The function returns either the name of the next node or `END`.

```
// After planner: go to runStep unless planning failed
function routeAfterPlanner(state: OState): "runStep" | typeof END {
  if (state.phase === "failed" || state.plan.length === 0) return END;
  return "runStep";
}

// After each step: loop back unless session is done/failed/aborted
function routeAfterStep(state: OState): "runStep" | typeof END {
  if (state.phase === "done" || state.phase === "failed" || state.aborted) return END;
  return "runStep";
}
```

This implements a **loop without explicit loop constructs** — a key LangGraph pattern. The node runs, updates state, then the router decides whether to re-enter the same node or end.

8

Graph Assembly & Compilation

```
const graph = new StateGraph(OrchestratorState)
  .addNode("planner", plannerNode)
  .addNode("runStep", runStepNode)
  .addEdge(START, "planner") // always enter planner first
  .addConditionalEdges("planner", routeAfterPlanner) // plan ok? → runStep, else END
  .addConditionalEdges("runStep", routeAfterStep); // more steps? → runStep, else END

const orchestratorGraph = graph.compile();
```

`graph.compile()` validates the graph topology (no unreachable nodes, all edges defined) and returns an executable runnable. The graph is invoked with:

```
const finalState = await orchestratorGraph.invoke(initialState, {
  recursionLimit: 64, // max node invocations before automatic stop
  signal,             // AbortSignal from HTTP request for cancellation
});
```

9 Database Schema for Agents

Five Prisma models back the agent orchestration system. All IDs are UUIDs; timestamps are managed by Prisma.

Model	Purpose	Key Fields
Machine	A remote server reachable over SSH	host, port, username, authType, sshCredential (AES-256 encrypted), status
Agent	An AI CLI tool running on a machine	machineId, agentType, agentCommand, idleMarker, status
AgentSession	One orchestration run (goal → plan → execution)	goal, supervisorModel, workingDirectory, plan (JSON), status, endedAt
AgentSessionAgent	Many-to-many: which agents participate in a session	sessionId, agentId
AgentMessage	Full message log of the orchestration session	sessionId, fromAgentId?, toAgentId?, role, content, messageType

The `plan` column on `AgentSession` is a `Json` field that stores the live `PlanStep[]` array. It is updated after every state transition so the UI can poll and display real-time progress.

10 API Surface

Route	Method	Purpose
<code>/api/agents/sessions/[id]/start</code>	POST	Invokes <code>runOrchestrator()</code> ; the graph runs in the background. Returns 202 immediately.
<code>/api/agents/sessions/[id]/state</code>	GET	Polls live session state (plan, messages, status) for UI updates.
<code>/api/agents/sessions</code>	POST	Creates a new AgentSession record and links selected agents.
<code>/api/agents</code>	GET / POST	CRUD for agent definitions.
<code>/api/agents/machines</code>	GET / POST	CRUD for machine registrations (SSH credentials stored encrypted).

11 Full-Stack Integration (Next.js App Router)

The orchestrator is wired end-to-end through Next.js 16's App Router:

- Server Components** (`src/app/(main)/agents/page.tsx`) fetch machines, agents, and sessions in parallel with `Promise.all`, then pass serialized data to a client component.
- Client Component** (`AgentsHubClient`) renders the UI: machine list, agent registry, session creation form, and live session detail view with step-by-step progress.
- Session detail page** (`src/app/(main)/agents/sessions/[id]/page.tsx`) polls `/api/agents/sessions/[id]/state` to show real-time plan execution status, message log, and per-step handoff notes.
- WebSocket terminal** (`src/lib/ws-terminal-server.ts`) manages active tmux sessions per agent, providing `sendToAgentInput`, `waitForIdleMarker`, and `captureTmuxPane` used by `runStepNode`.

FIGURE SET A — AGENTS HUB (MACHINES, AGENTS, SESSIONS)

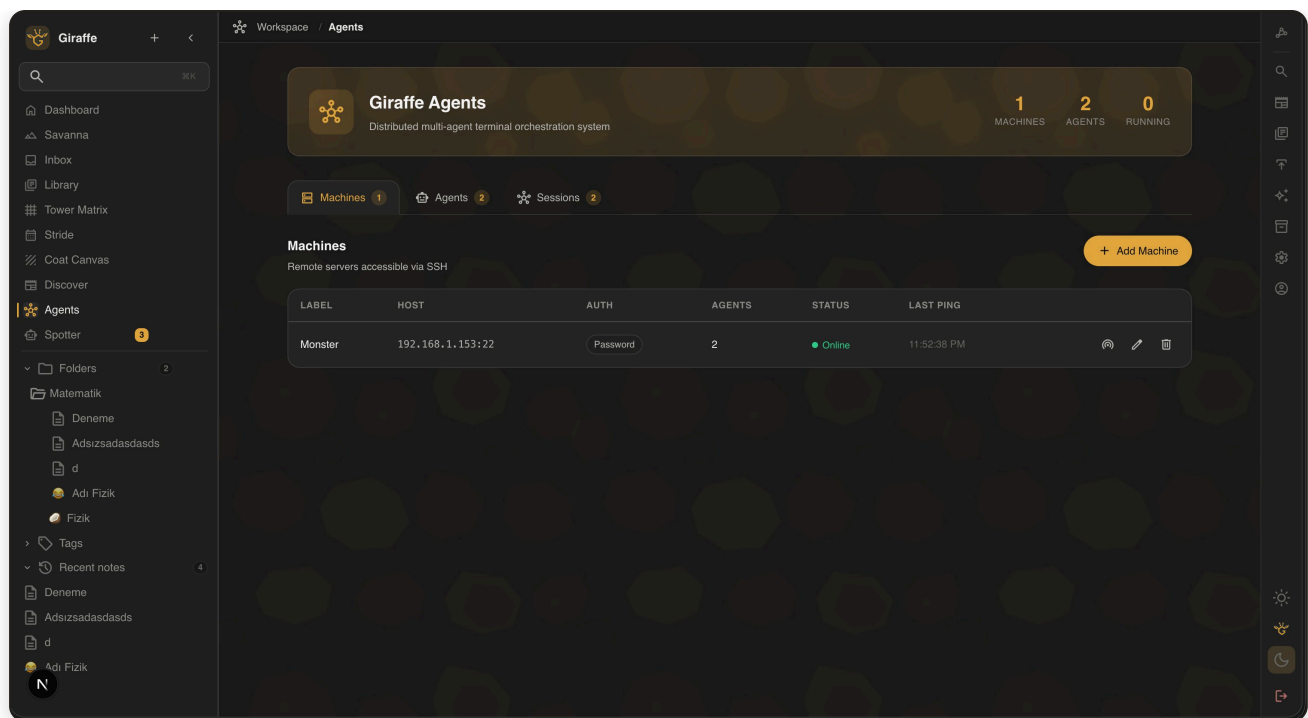


Figure 1 — Machines tab.

Remote servers accessible over SSH. The Monster machine (192.168.1.153:22) is registered with password auth, currently Online, and runs 2 agents. Credentials are stored AES-256 encrypted in the database.

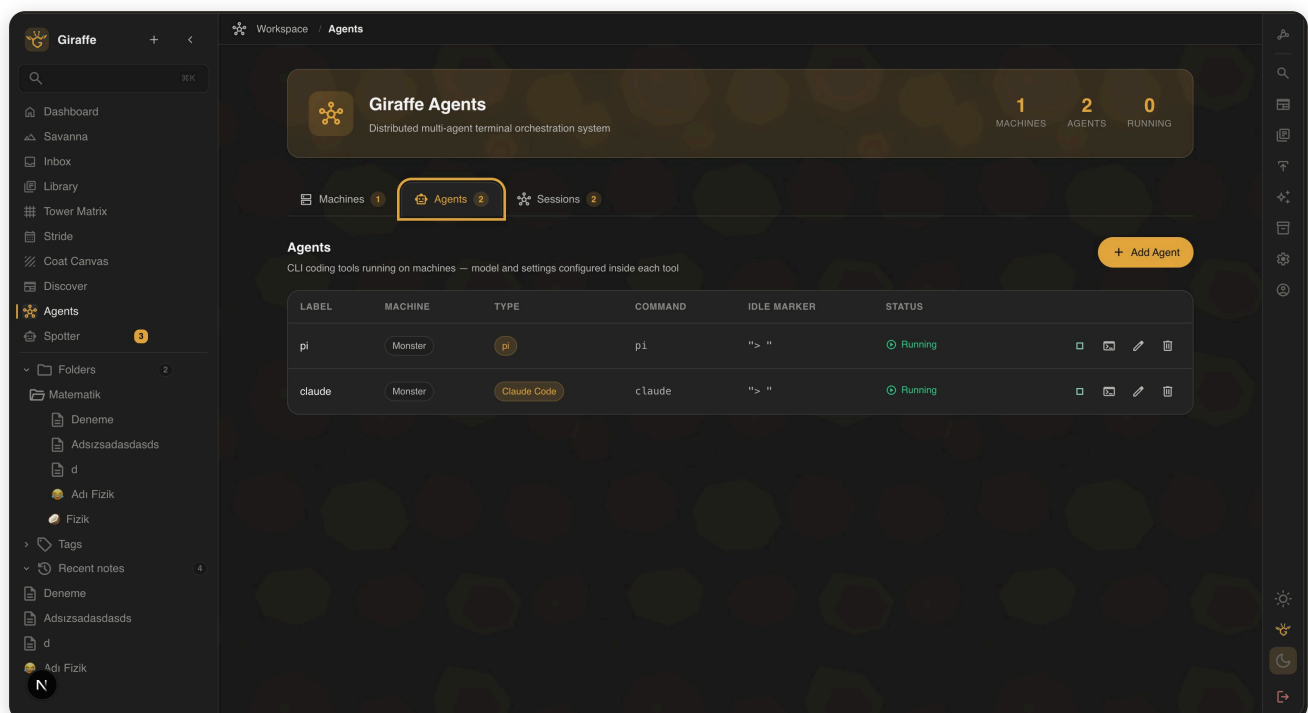


Figure 2 — Agents tab.

CLI coding tools running on the Monster machine. Two agents are registered — `pi` and `claude` (Claude Code) — each with their start command and idle marker (`"> "`) used by `waitForIdleMarker()` to detect completion.

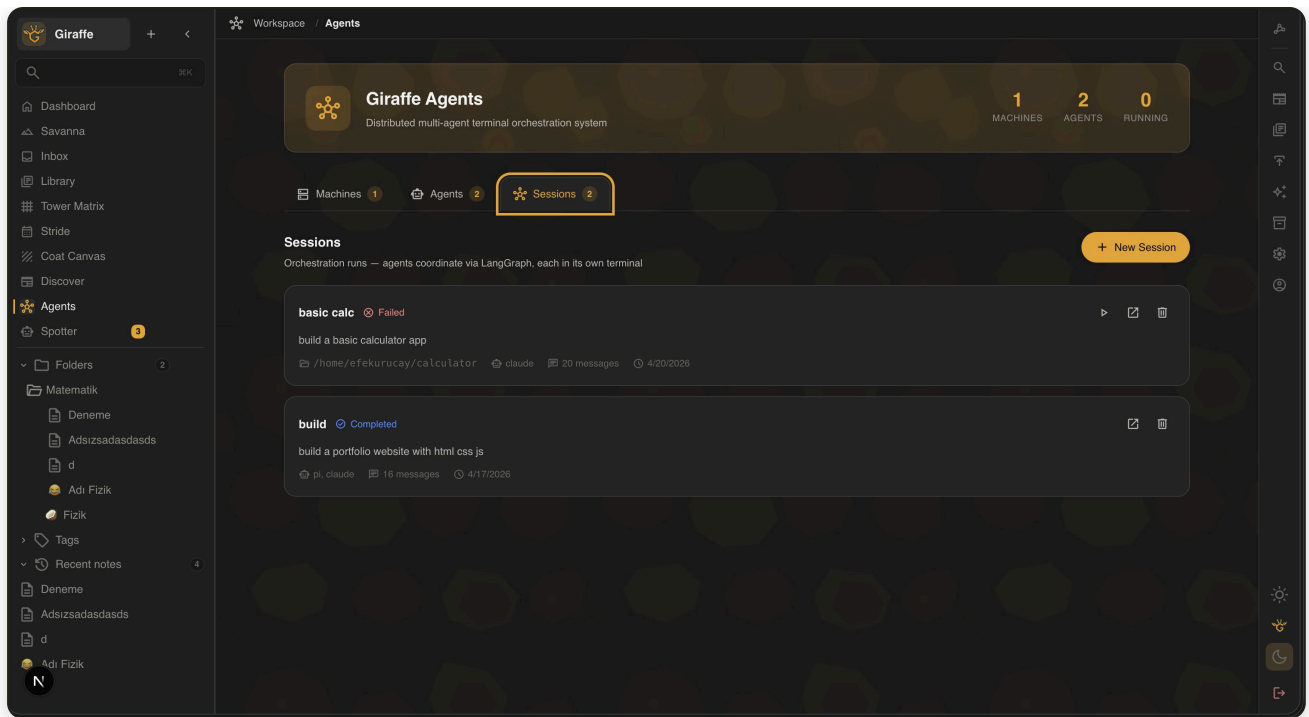


Figure 3 — Sessions tab.

Orchestration runs coordinated via LangGraph. Visible: a prior build session (Completed, 16 messages, 2 agents) and a basic calc session (Failed) used as the running example for Figure Set B.

FIGURE SET B — SESSION LIFECYCLE (CREATION → PLAN → EXECUTION)

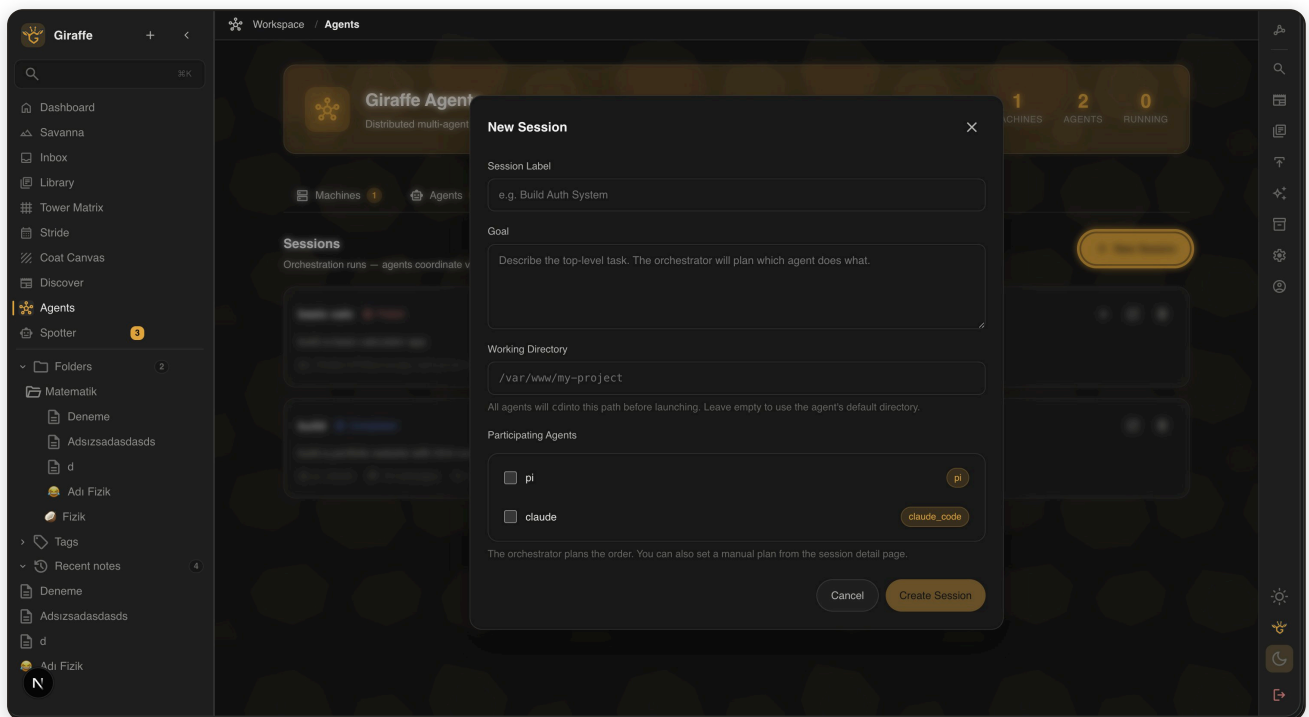


Figure 4 — New Session dialog.

The user provides a session label, a natural-language goal, a working directory (passed as `workingDirectory` into the LangGraph state), and picks participating agents. The orchestrator will plan the step order from this input.

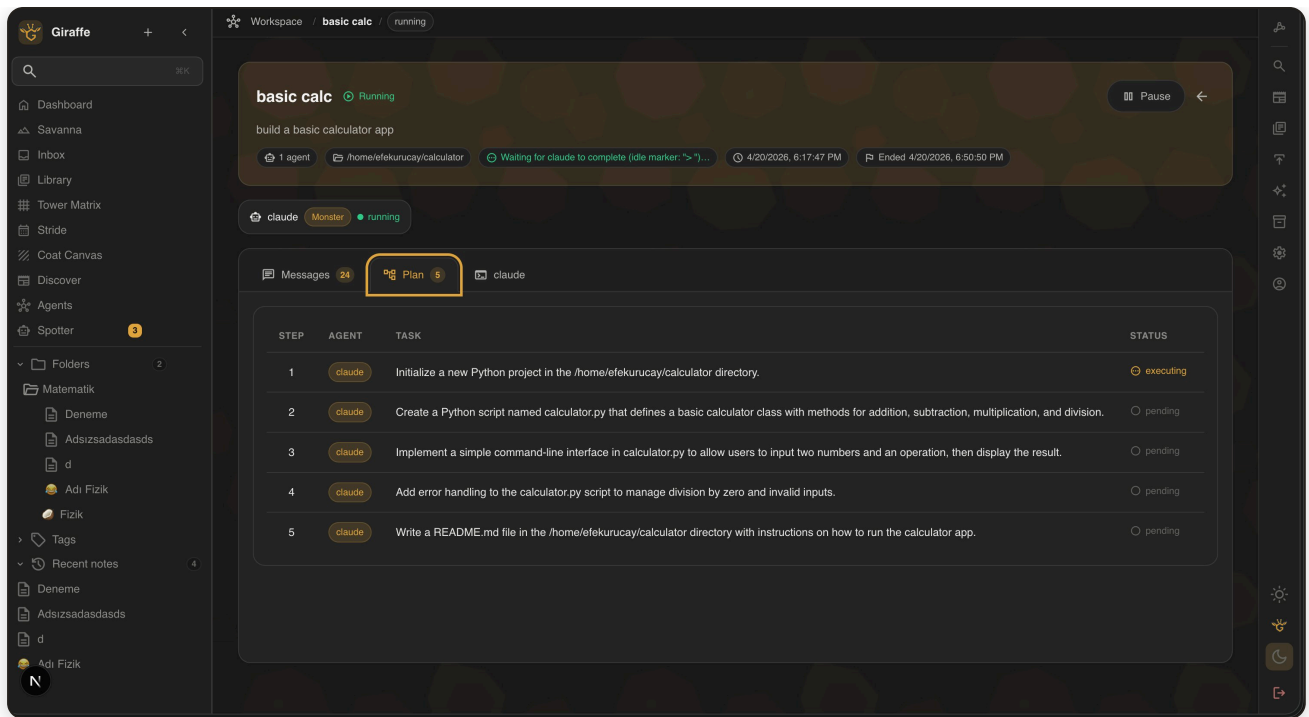


Figure 5 — Plan tab (live step progress).

The planner node produced a 5-step plan for the goal "build a basic calculator app". Step 1 is *executing*; steps 2–5 are *pending*. This view polls `/api/agents/sessions/[id]/state` to reflect state updates written by each LangGraph transition.

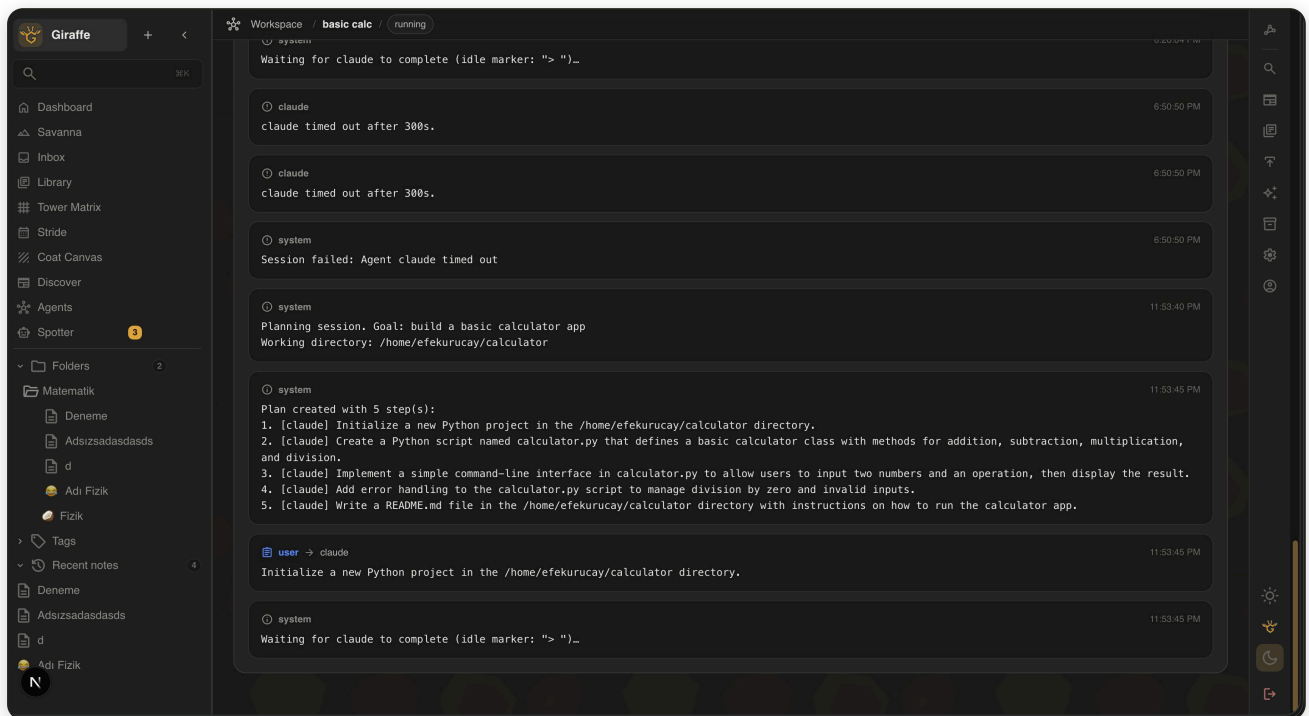


Figure 6 — Messages tab (orchestrator event log).

The full `AgentMessage` log for the session: planner output, task dispatch, and — importantly — the 300-second idle timeout that caused `runStepNode` to mark the step *failed*, setting `phase = "failed"` and routing the graph to `END`. This illustrates the error-handling path described in Section 7.

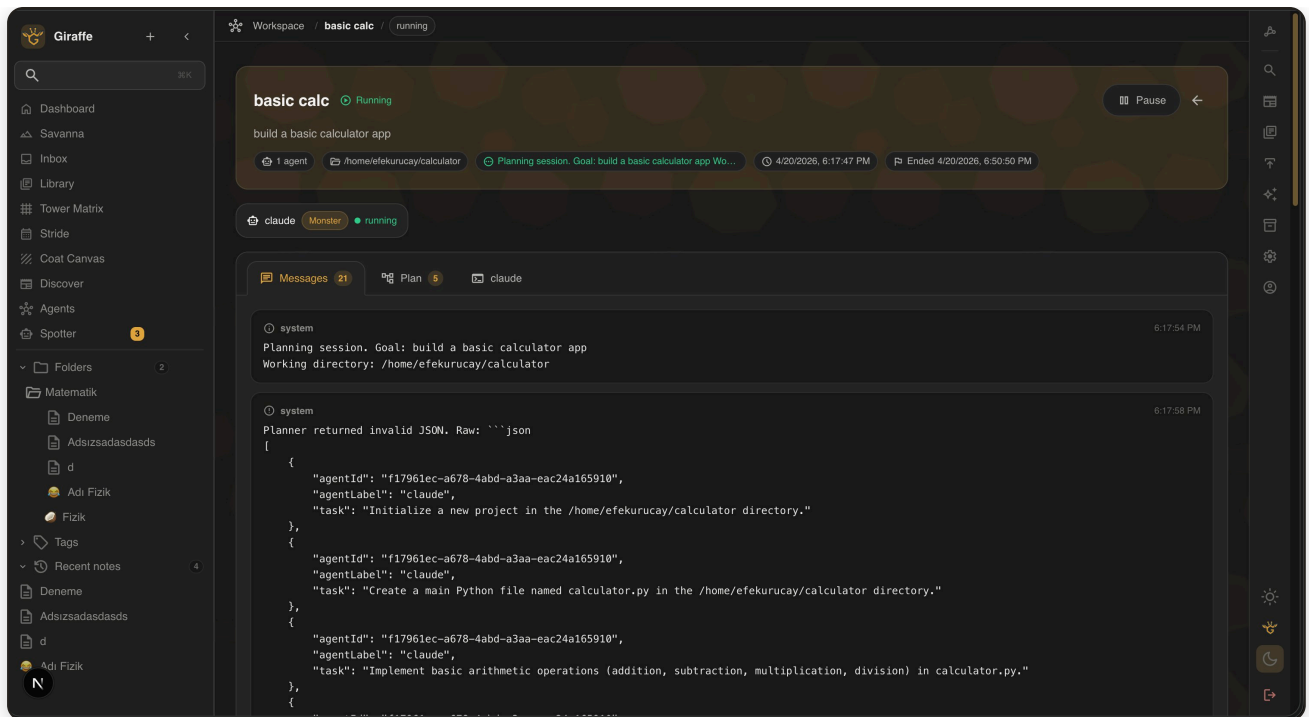


Figure 7 — Planner resilience: invalid JSON recovery.

A second failure mode: the planner LLM returned JSON wrapped in markdown fences (````json`). The orchestrator logs the raw response, runs its cleanup regex, re-parses successfully, and proceeds. This is why `plannerNode` strips fences before `JSON.parse` rather than trusting the model's instruction-following.

12 Features Implemented

LangGraph Orchestration Engine

- Typed state machine with `StateGraph`, `Annotation.Root`, conditional edges, and `END` / `START` sentinels
- Two-node pipeline: `planner` (LLM planning) → `runStep` (loop)
- Conditional routing based on runtime state (`phase`, `plan.length`, `aborted`)
- Recursion-limited execution with `AbortSignal` support for cancellation
- Agent handoff notes using structured delimiters (`[HANDOFF_START]...[HANDOFF_END]`)
- 5-minute per-step idle timeout; 2-minute handoff timeout
- Live plan persistence to PostgreSQL after each state transition
- Full message log (`AgentMessage`) for every event: task dispatch, agent response, handoff, error, completion

Agent & Machine Registry

- Register remote machines with SSH credentials (stored AES-256 encrypted)
- Register AI agents (Claude Code, Aider, custom) with configurable idle markers

- Machine health / connectivity status tracking

Session Management

- Create sessions with a goal, supervisor model selection, and agent roster
- Start / stop / view sessions
- Real-time plan view (step index, status per step, handoff notes)

Terminal Infrastructure

- SSH connection management per machine
- tmux session lifecycle per agent (`new-session` , `send-keys` , `capture-pane`)
- Idle detection by polling for the agent's configured prompt marker
- Handoff note extraction by regex over captured terminal output

Knowledge Editor (broader product context)

- Block-based editing with Tiptap (callout, toggle, wikilink, slash commands)
- Canonical AST storage in PostgreSQL (`Note` + `Block` tables)
- Wikilinks, backlinks, graph projection, tag extraction
- Markdown & MDX export, public note routes, folder hierarchy
- Docker Compose production stack with multi-arch image publishing via GitHub Actions

13 Setup & Running the Project

Prerequisites

- Node.js 20+
- Docker & Docker Compose
- An OpenAI API key (for the orchestrator's planner LLM)
- At least one remote machine with SSH access and a supported AI CLI tool installed

Step-by-Step Local Setup

```
# 1. Clone the repository
git clone https://github.com/GiraffeGraph/giraffe.git
cd giraffe

# 2. Install dependencies
npm install

# 3. Start PostgreSQL
docker compose up -d

# 4. Configure environment
cp .env.example .env
# Edit .env – required values:
#   DATABASE_URL=postgresql://postgres:postgres@localhost:5432/giraffe
#   AUTH_SECRET=<random-64-char-string>
#   NEXTAUTH_URL=http://localhost:3000
#   OPENAI_API_KEY=sk-...

# 5. Run database migrations
npx prisma migrate dev

# 6. Start the development server
npm run dev
```

Open `http://localhost:3000`. Register an account, then navigate to **/agents** to register machines and agents.

Running the Verification Suite

```
# Type-check, lint, unit tests, and production build
npm run verify

# Unit tests only
npm run test:run

# Coverage report
npm run test:coverage
```

Production Deployment (Docker)

```
cp .env.production.example .env.production
# Edit .env.production
./scripts/prod-up.sh
```

Step-by-Step: Using the LangGraph Orchestrator

1. Go to **/agents** → "Machines" tab → Add a machine (host, SSH credentials).
2. Go to "Agents" tab → Add an agent (link to a machine, set the agent command and idle marker).
3. Go to "Sessions" tab → Create a new session (enter a goal, choose agents, pick supervisor model).
4. Click **Start**. The LangGraph state machine invokes the planner (calls GPT-4o), produces a plan, then executes each step sequentially.
5. Watch live progress in the session detail page — each step shows its status and the handoff note passed to the next agent.

14 Git & Version Control

Git Repository

<https://github.com/GiraffeGraph/giraffe>

Selected commits relevant to the LangGraph feature:

SHA	Message
dc8a509	feat(agents): migrate to Smart Orchestrator, Dumb Agents via LangGraph
efdbd46	(fix) tmux key name error
5239a3a	(fix) json parse error

The project follows a release tagging convention (`VMAJOR.MINOR.PATCH`) with GitHub Actions running quality checks (lint, typecheck, tests, build), a smoke test against the real Docker Compose stack, and Docker Hub image publishing on every passing release tag.

15 Reflection & Discussion Points

What LangGraph enables that a plain function chain cannot

- **Typed, inspectable state:** every field has a declared reducer and a default. Debugging is easier because state is explicit.
- **Runtime branching:** the router function reads live state, so decisions like "abort if timed out" require no special framework plumbing.

- **Natural loops:** the `runStep → runStep` self-loop with a terminal condition is a first-class graph pattern, not a workaround.
- **Cancellation:** passing an `AbortSignal` into `graph.invoke()` propagates cancellation automatically across all async node invocations.

Design decisions and trade-offs

- **Replace-on-write reducers** for `plan` and `agentRegistry`: these are always written as complete arrays, so a simple last-write-wins reducer is correct and prevents accidental partial merges.
- **Polling vs. streaming:** the orchestrator polls the terminal for an idle marker rather than streaming token output. This is simpler, works with any CLI tool, and avoids WebSocket complexity inside the graph.
- **Out-of-band persistence:** the graph calls `updateAgentSessionPlan()` directly inside nodes rather than using LangGraph checkpointers. This keeps the PostgreSQL schema fully under the application's control.
- **Temperature 0.1 for the planner:** low temperature makes the JSON output more deterministic and reduces parse failures.

Possible extensions

- Add a **validator node** after `runStep` that checks test results before advancing.
- Use **LangGraph checkpointers** (PostgreSQL adapter) to persist graph state between server restarts.
- Support **parallel step execution** using LangGraph's `Send` API when steps have no dependencies.
- Add a **human-in-the-loop** pause using LangGraph's interruption / approval mechanism.

Summary

Giraffe demonstrates LangGraph's core strengths — typed state, conditional routing, and node loops — in a real production context: coordinating multiple AI coding agents across remote machines. The graph is small (two nodes) but the pattern it implements — plan then iteratively execute — is directly applicable to any multi-agent workflow.